



**UNIVERSIDADE
ESTADUAL DO CEARÁ**

**Centro de Ciências e Tecnologia, CCT
Bacharelado em Ciência da Computação
Universidade Estadual do Ceará - UECE**

**EXTENSÃO III
Relatório final - FoodGuard**

**GABRIEL PINHEIRO REZENDE DE LIMA
ARISTÓTELES FEITOSA
PAULO MATEUS BARROSO DE VASCONCELOS
VICTOR MANOEL MAGALHÃES TIMBÓ
PEDRO OTÁVIO DE SOUSA BEZERRA**

FORTALEZA-CE

1. Introdução.....	3
2. PARTE I — Sprint 1.....	3
2.1 Definição do Projeto.....	3
2.2 Definição de Arquitetura.....	3
2.3 Definição de Requisitos.....	4
2.4 Prototipação da Aplicação.....	5
2.5 Início da Implementação.....	5
2.6 Organização da Equipe.....	5
2.7 Dificuldades.....	5
2.8 Próximos Passos.....	6
3. PARTE II — Sprint 2.....	6
3.1 Desenvolvimento do Projeto.....	6
3.2 Integração Backend e Frontend.....	7
3.3 Garantia de Qualidade e Engenharia de Prompt.....	7
3.4 Dificuldades.....	7
3.5 Próximos Passos.....	7
4. PARTE III — [⚠ DATA NÃO INFORMADA].....	7
4.1 O que foi feito.....	7
4.2 Decisões Finais.....	8
4.3 Dificuldades.....	8
5. Próximos Passos — Além da Cadeira de EX III.....	8
6. Considerações Finais.....	8

1. Introdução

Este relatório apresenta a trajetória de desenvolvimento do projeto FoodGuard, detalhando desde a definição inicial da proposta até as versões mais recentes da aplicação. Ao longo do documento, são descritas as decisões técnicas, os desafios enfrentados, as soluções adotadas e as evoluções obtidas nas diferentes fases do projeto.

O objetivo é registrar, de forma organizada e objetiva, todo o processo de construção da plataforma, desde os estudos iniciais sobre requisitos e arquitetura, passando pelo desenvolvimento do frontend e backend, pela integração com modelos de linguagem (LLMs), até as melhorias implementadas e os próximos passos previstos.

O FoodGuard é uma plataforma web vinculada à disciplina de Extensão III que tem como propósito auxiliar usuários na identificação de componentes alergênicos em produtos alimentícios, interpretando rótulos via código de barras ou comandos de texto e cruzando os dados nutricionais com o formulário de saúde do usuário.

2. PARTE I — Sprint 1

2.1 Definição do Projeto

O projeto FoodGuard nasceu da necessidade de identificar componentes alergênicos em produtos alimentícios, prevenindo reações adversas nos consumidores. A solução escolhida foi uma plataforma web vinculada à disciplina de Extensão III, capaz de interpretar rótulos via código de barras ou comandos de texto, cruzando os dados nutricionais com o formulário de saúde (perfil de saúde) do usuário.

As principais delimitações de escopo definidas nesta sprint foram:

Plataforma Alvo: Estritamente web. Não haverá desenvolvimento de aplicativo mobile. Responsividade não é foco no momento e será tratada em etapas futuras, se houver tempo.

MVP: Optou-se por não definir um MVP fechado durante esta sprint, priorizando apenas o levantamento das necessidades e a criação de uma fundação tecnológica básica.

2.2 Definição de Arquitetura

A arquitetura foi dividida entre o servidor (Python/Django) e o cliente (React), com foco, nesta sprint, apenas na instalação e configuração inicial das ferramentas, sem implementações robustas de lógica de negócio ou segurança.

Backend (API):

Desenvolvido em Python utilizando o framework Django (v4.2.16) e Django REST Framework para construção das APIs.

O servidor rodará de forma assíncrona utilizando Uvicorn (uvicorn[standard], uvicorn-worker).

Autenticação de sessões e rotas preparada com django-allauth[mfa] e PyJWT.

Bibliotecas de suporte adicionadas: Pillow (processamento de imagens), hiredis (comunicação com Redis) e reportlab.

Ferramentas de customização do painel administrativo: django-unfold, django-admin-autocomplete-filter, django-crispy-forms e crispy-tailwind.

Nota de Segurança: O pacote argon2-cffi foi adicionado, mas a decisão técnica sobre criptografia de credenciais e dados médicos foi postergada para sprints futuras. Documentação das rotas será feita com drf-spectacular.

Frontend (Interface Web):

Construído em React (v19) com roteamento gerenciado pelo react-router-dom.

Consumo da API Django via axios; gerenciamento de estados assíncronos e cache com @tanstack/react-query.

Interface de usuário (UI) com Tailwind CSS (via tailwind-merge e clsx) em conjunto com componentes do Radix UI e Shadcn.

Iconografia: pacotes lucide-react e react-icons; tipografia padrão com a fonte @fontsource-variable/geist.

Utilitários adicionais: date-fns e react-day-picker para manipulação de datas futuras.

2.3 Definição de Requisitos

Foi estruturado o documento formal "FoodGuard Requisitos - Extensão 3.pdf", contendo o mapeamento completo das funcionalidades do sistema. Os levantamentos realizados identificaram:

17 Requisitos Funcionais, abrangendo fluxos desde o cadastro do formulário de saúde até a edição do prompt de chat e exclusão de conversas.

4 Requisitos de Qualidade, voltados à usabilidade, desempenho e segurança da aplicação.

14 Regras de Negócio, como a proteção de dados na comunicação com a LLM e a obrigatoriedade do formulário de saúde para liberar o acesso ao chat.

2.4 Prototipação da Aplicação

O desenho das interfaces visuais foi iniciado para refletir os requisitos de login, cadastro, formulário de saúde e a tela principal de interação com o chatbot. A página "Home", que conteria as seções informativas "Quem somos" e "Como funciona", foi removida do escopo de prototipação desta sprint para que o esforço ficasse concentrado nos fluxos essenciais de autenticação e chat.

<https://www.figma.com/design/R7UL1Wfz9dV9JUQbRCO8np/Alergia?node-id=6-364&t=6jiIjHKWijv0d02v-1>

2.5 Início da Implementação

Foram realizados os seguintes avanços técnicos nesta sprint:

API para Leitura de Código de Barras: Iniciados estudos e testes preliminares, de forma não robusta, com a biblioteca zbar-wasm para extração do código de barras de imagens. O planejamento contempla o uso futuro do modelo YOLOv8n integrado com ONNX Runtime-web para processar a visão computacional localmente no navegador.

Frontend: O repositório frontend foi inicializado com todas as dependências listadas na arquitetura. O foco esteve apenas em estabelecer o esqueleto do projeto — configuração do Tailwind e arquivos de roteamento do React Router —, sem a construção das páginas em si.

2.6 Organização da Equipe

A distribuição de responsabilidades entre os membros da equipe nesta sprint foi organizada da seguinte forma:

Gabriel Pinheiro, Paulo Matheus e Pedro Otávio: Responsáveis pela elaboração da documentação e pelo levantamento e estruturação dos requisitos do sistema.

Victor Manoel: Responsável pela prototipação das interfaces. A validação do artefato contou com a participação ativa de todos os integrantes.

Aristóteles Feitosa: Responsável por definir e estruturar a arquitetura técnica do projeto no backend e sua conexão com o frontend.

2.7 Dificuldades

As principais dificuldades encontradas nesta sprint foram:

Clareza na Documentação: Dificuldade na construção e redação dos requisitos do sistema, exigindo esforço adicional para escrevê-los da forma mais clara e sem ambiguidades.

Coleta de Dados de Saúde: Definição complexa sobre o nível de detalhes e quais informações específicas seriam coletadas no formulário de saúde, equilibrando eficácia do modelo com usabilidade e privacidade do usuário.

Questões Éticas: Definição do limiar até onde o chat pode auxiliar os usuários sem comprometer ou invalidar a importância de um diagnóstico ou acompanhamento profissional de saúde.

Definição da Plataforma Alvo: Discussões prolongadas para decidir sobre a viabilidade de construir uma versão mobile ou manter o foco exclusivo no ambiente web.

Integração de Inteligência Artificial: Dificuldade na decisão arquitetural sobre a utilização ou não de RAG (Retrieval-Augmented Generation) para embasar as respostas do chatbot com o contexto do usuário.

2.8 Próximos Passos

Avançar na construção das páginas do frontend, saindo apenas da configuração inicial do Tailwind e arquivos de roteamento do React Router estabelecidos na sprint atual.

Definir a decisão técnica sobre como a criptografia das credenciais e dos dados médicos dos usuários será efetivamente configurada.

Avançar com os testes preliminares da biblioteca zbar-wasm e planejar o uso futuro do modelo Yolov8n integrado com ONNX Runtime-web para o processamento de visão computacional localmente.

3. PARTE II — Sprint 2

3.1 Desenvolvimento do Projeto

Com os requisitos principais estabelecidos na Sprint 1, o foco desta sprint foi no desenvolvimento das interfaces de interação e na estruturação das portas de entrada da aplicação para o usuário.

As principais entregas do lado do frontend foram:

Interface do Chat e Navegação: Implementação da tela de conversa principal com a LLM, com suporte contínuo ao envio de texto. O histórico de conversas passou a ser visível ao usuário, e foi adicionado mecanismo de navegação para iniciar novas conversas de forma limpa.

Upload de Mídias: Funcionalidade de envio de imagens desenvolvida e integrada ao campo de envio do prompt.

Autenticação: Iniciada a estruturação das telas de Login e Cadastro, preparando o terreno para as lógicas de segurança.

Design & UI: Definidos o layout e a identidade visual da Landing Page do projeto.

3.2 Integração Backend e Frontend

A conexão síncrona entre o frontend e o backend foi estabelecida com sucesso no escopo atual da sprint. A integração garante que a interface desenvolvida consiga se comunicar adequadamente com os serviços da API da aplicação.

Foram implementados os endpoints da API REST referentes aos domínios de conversa (listagem, envio de mensagens e exclusão de chats) e do formulário de saúde (criação, consulta e atualização).

A persistência de dados foi consolidada integrando o PostgreSQL 16 via psycopg, com a modelagem das entidades centrais da aplicação (Chat, Message e formulário de saúde).

A documentação automática da API foi gerada via OpenAPI/Swagger utilizando a biblioteca drf-spectacular.

Para o núcleo de IA, o modelo gemini-2.5-flash foi encapsulado em um serviço dedicado (GeminiClient). O fluxo de mensagens exclui deliberadamente dados pessoais como nome e e-mail antes do envio à LLM para preservar a privacidade do usuário.

A comunicação com a LLM utilizou DSPy (Engenharia de Prompt Programática) com saída estruturada via Pydantic, garantindo uma integração confiável e eliminando o parsing frágil de texto livre.

3.3 Garantia de Qualidade e Engenharia de Prompt

Foram realizados testes e definições essenciais para garantir o funcionamento correto do cruzamento de informações com a LLM:

Planejamento de Testes: Criação de uma planilha matriz de casos de teste, estruturada para cobrir possíveis variações de tipos de usuários, formatos de imagens enviadas e diversidade de prompts.

Arquitetura de Dados: Definição do formato padrão do prompt (payload de envio) e da estrutura de retorno que a resposta da LLM deve adotar para ser exibida corretamente ao usuário.

3.4 Dificuldades

A equipe enfrentou desafios que foram além da codificação, envolvendo também o alinhamento do produto e a comunicação interna:

Definição Ética e de Produto da IA: Um grande desafio não técnico foi estabelecer o limite de atuação do chatbot. A equipe precisou debater e parametrizar as regras de operação da IA para garantir um caráter estritamente informativo e a proibição de prescrições médicas. Foi necessário alinhar uma postura conservadora anti-alucinação, instruindo o modelo a classificar o alimento como "DANGEROUS" em caso de dúvida.

Comunicação e Alinhamento de Integração: O estabelecimento da conexão entre as frentes de trabalho exigiu uma comunicação constante para definir a arquitetura de dados. O time precisou alinhar o formato padrão da carga de envio (payload) e a estrutura exata de retorno da IA para que o frontend conseguisse renderizar as informações corretamente para o usuário.

Elaboração e Padronização de Testes: A criação da matriz de casos de teste demandou um forte esforço analítico e de planejamento da equipe, pois foi necessário prever múltiplas variações de perfis de usuários, formatos de imagens recebidas e uma grande diversidade de prompts para garantir a segurança alimentar.

Estruturação das Respostas da LLM (Desafio Técnico e de Gestão): O desafio inicial de evitar o parsing frágil de texto livre gerado pelo modelo exigiu tempo de pesquisa e adaptação do grupo. Esse obstáculo foi superado coletivamente com o uso da biblioteca DSPy, garantindo a serialização confiável em JSON e estabilizando a aplicação.

3.5 Próximos Passos

Implementar as lógicas de segurança de forma definitiva nas telas de Login e Cadastro, cujo pontapé inicial de estruturação já foi dado.

Finalizar as regras de domínio e exceções customizadas para garantir o fluxo completo de invalidação de chats e bloqueio de mensagens conforme o preenchimento do formulário de saúde.

4. PARTE III — Sprint 3

4.1 O que foi feito

Nesta etapa de finalização, os esforços concentraram-se em concluir os fluxos de segurança, amarrar as regras de negócio no backend e refinar a experiência do usuário (UX) estabelecidos nas sprints anteriores. As principais entregas foram:

Frontend e UX: Conclusão do desenvolvimento das rotas de Autenticação (Login, Cadastro e Recuperação de Senha) e implementação das interfaces e modais para visualização e edição

de dados do perfil e do formulário de anamnese. Foram padronizados os indicadores de carregamento (loading) e alertas visuais (toasts) para orientar o usuário em cenários de erro e processamento.

Backend e Regras de Negócio: Aplicação de segurança definitiva com validação rigorosa de tokens de sessão nas rotas da API via Django REST Framework. Foi implementada a lógica de invalidação de contexto, que bloqueia conversas antigas caso o usuário edite sua anamnese, exigindo a abertura de um novo chat para manter a consistência e segurança das análises.

Visão Computacional e APIs: Consolidação do uso da biblioteca zbar-wasm para extração de códigos de barras a partir das imagens enviadas pelo usuário. O pipeline de inteligência artificial foi finalizado, integrando a busca de ingredientes no OpenFoodFacts com os modelos GPT-4o e GPT-4o-mini estruturada via DSPy.

Testes e Qualidade (QA): Execução da validação de ponta a ponta (End-to-End), garantindo que o fluxo completo — desde a obrigatoriedade da anamnese até o retorno estruturado da LLM — ocorra sem falhas críticas, cobrindo as variações da matriz de testes.

4.2 Decisões Finais

Durante o fechamento da aplicação, as seguintes decisões técnicas e de produto foram firmadas:

Privacidade de Dados Assegurada: Para cumprir rigorosamente os requisitos de privacidade, definiu-se que o sistema de roteamento interno deve sempre mascarar ou remover os campos identificadores (Nome, E-mail, Data de Nascimento) antes de enviar qualquer contexto clínico para o modelo de IA.

Fluxo de Fallback para Imagens: Optou-se por implementar um tratamento de erro amigável para a leitura de imagens. Caso a foto do código de barras esteja ilegível (ex: desfoque), o processamento local é abortado e o sistema orienta o usuário a digitar a requisição, mantendo-o engajado na plataforma.

Prevenção contra Alucinações: Consolidou-se a regra na qual o modelo de IA atua de forma estritamente conservadora. Na falta de clareza sobre um ingrediente retornado pela API ou analisado no prompt, o alimento é sinalizado como inadequado ("DANGEROUS"), priorizando a saúde e segurança do usuário.

4.3 Dificuldades

As dificuldades nesta reta final estiveram mais atreladas à comunicação com serviços de terceiros e à performance da aplicação:

Inconsistência de Dados Externos: A equipe enfrentou dificuldades ao lidar com dados faltantes ou formatações inconsistentes oriundos da API do OpenFoodFacts. Foi necessário

desenvolver tratamentos adicionais no backend para que esses dados irregulares não quebrassem a estrutura de resposta em JSON esperada pela LLM.

Desempenho da Visão Computacional Local: Otimizar o tempo de extração do código de barras via zbar-wasm no próprio navegador provou-se desafiador. Houve esforço focado em garantir que esse processamento não travasse a interface (UI) e respeitasse os requisitos de tempo máximo de resposta da plataforma estipulados nos requisitos de qualidade.

5. Próximos Passos — Além da Cadeira de EX III

Responsividade: Implementação da responsividade da interface para adequação a diferentes dispositivos, visto que não foi o foco principal nas etapas iniciais.

Versão Mobile: Retomada das discussões sobre a viabilidade e o desenvolvimento de um aplicativo mobile nativo, uma vez que o escopo atual foi estritamente voltado para a plataforma Web.

Segurança Avançada: Implementação efetiva da criptografia de credenciais e dados médicos, cuja decisão arquitetural havia sido postergada.

Visão Computacional Aprimorada: Uso do modelo Yolov8n integrado com ONNX Runtime-web para processar a extração de código de barras e visão computacional de forma robusta e local.

Aprimoramento de IA: Decisão arquitetural final sobre a utilização de RAG (Retrieval-Augmented Generation) para embasar ainda mais as respostas do chatbot com o contexto médico do usuário.

6. Considerações Finais

O projeto FoodGuard representou uma experiência enriquecedora para toda a equipe. Ao longo do desenvolvimento para a disciplina de Extensão III, foram aplicados conceitos técnicos de engenharia de software e assimiladas práticas essenciais de gestão de projeto, colaboração e solução de problemas em um ambiente de trabalho real.

Do ponto de vista técnico, a equipe aprimorou habilidades em desenvolvimento Full Stack — utilizando Django no backend e React no frontend —, fortaleceu o domínio sobre controle de versão com Git e GitHub, e avançou no entendimento de integração com modelos de linguagem (LLMs) e processamento de visão computacional. O sistema consolida-se como um auxílio inovador para a identificação de alimentos com potenciais alergênicos, aliando tecnologia à prevenção de reações adversas na saúde dos usuários.

